

./media/logo-upt.png

Figure 1: ./media/logo-upt.png

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Proyecto de Antivirus

Curso: *Calidad y Pruebas de Software*

Docente: *Mag. Patrick Cuadros Quiroga*

Integrantes:

LLica Mamani, Jimmy Mijair (2023076789)

Sierra Ruiz, Iker Alberto (2023077090)

Tacna – Perú

2026

Sistema *SecureGuard Antivirus*

Informe de Factibilidad

Versión *1.0*

**CONTROL DE
VERSIONES**

Versión	Hecha por	Revisada por	Aprobada por	Fecha
1.0	LLica Mamani, Jimmy Mijair	Sierra Ruiz, Iker Alberto	LLica Mamani, Jimmy Mijair	28/03/2026

INDICE GENERAL

1. Descripción del Proyecto
2. Riesgos
3. Análisis de la Situación actual
4. Estudio de Factibilidad
 - 4.1 Factibilidad Técnica

- 4.2 Factibilidad económica
- 4.3 Factibilidad Operativa
- 4.4 Factibilidad Legal
- 4.5 Factibilidad Social
- 4.6 Factibilidad Ambiental
- 5. Análisis Financiero
 - 5.1 Justificación de la Inversión
 - 5.1.1 Beneficios del Proyecto
 - 5.1.2 Criterios de Inversión
- 6. Conclusiones

Informe de Factibilidad

1. Descripción del Proyecto

1.1. Nombre del proyecto

SecureGuard Antivirus

1.2. Duración del proyecto

El proyecto tendrá una duración estimada de 16 semanas, distribuidas en 5 fases conforme al cronograma establecido en los issues del proyecto:

- Fase 1: Análisis y Fundamentos (3 semanas)
- Fase 2: Diseño de Arquitectura y Calidad (3 semanas)
- Fase 3: Desarrollo de Código y Construcción (5 semanas)
- Fase 4: Pruebas Dinámicas y Verificación (3 semanas)
- Fase 5: Gestión de Calidad y Cierre (2 semanas)

1.3. Descripción

El proyecto SecureGuard Antivirus consiste en el desarrollo de un software de seguridad informática especializado en la detección, prevención y eliminación de malware. Este sistema permitirá proteger los equipos de los usuarios finales mediante un motor de escaneo basado en firmas, monitoreo en tiempo real del sistema de archivos y una interfaz intuitiva que facilite la interacción con el usuario.

La importancia del proyecto radica en la creciente necesidad de proteger los sistemas informáticos contra amenazas como virus, gusanos, troyanos y ransomware. El contexto actual de transformación digital hace que la seguridad informática sea un aspecto crítico tanto para usuarios domésticos como para organizaciones.

1.4. Objetivos

1.4.1 Objetivo general Desarrollar un software antivirus funcional que permita detectar, prevenir y eliminar amenazas informáticas mediante un motor de escaneo basado en firmas y monitoreo en tiempo real, cumpliendo con los estándares de calidad establecidos en la norma ISO/IEC 25010.

1.4.2 Objetivos Específicos

Código	Objetivo Específico	Descripción de logro
OE-01	Implementar un motor de escaneo eficiente	Se logrará desarrollar la lógica para la lectura de archivos y comparación de patrones contra una base de datos de firmas de virus.
OE-02	Diseñar e implementar una base de datos de amenazas	Se logrará crear un esquema SQL para almacenar y gestionar las firmas de virus, permitiendo su actualización y consulta eficiente.
OE-03	Desarrollar un sistema de monitoreo en tiempo real	Se logrará programar servicios que detecten cambios en el sistema de archivos de forma activa, alertando al usuario ante posibles amenazas.
OE-04	Construir una interfaz de usuario intuitiva	Se logrará crear una consola o ventana gráfica que permita la interacción amigable con el usuario final.
OE-05	Implementar pruebas de calidad según ISO/IEC 25010	Se logrará medir y garantizar atributos de calidad como adecuación funcional, fiabilidad y mantenibilidad.
OE-06	Documentar y gestionar la configuración del proyecto	Se logrará establecer un repositorio en GitHub con la estructura adecuada y control de versiones para el trabajo colaborativo.

2. Riesgos

Señale los riesgos que pudieran afectar el éxito del proyecto.

Código	Riesgo	Descripción	Impacto	Probabilidad	Estrategia de Mitigación
R-01	Falsos positivos en el escaneo	El motor de escaneo podría identificar archivos legítimos como maliciosos, afectando la confianza del usuario.	Alto	Media	Implementar pruebas unitarias rigurosas (Issue #12) y curaduría manual de la base de datos de firmas (Issue #7).
R-02	Alto consumo de recursos	El monitoreo en tiempo real podría consumir excesiva CPU o RAM, degradando el rendimiento del equipo.	Alto	Media	Establecer métricas de eficiencia en el modelo de calidad (Issue #6) y optimizar el código mediante benchmarking.
R-03	Desviación del alcance	Incorporación de funcionalidades no planificadas que retrasen el cronograma.	Medio	Alta	Apegarse estrictamente a la visión y alcance definido (Issue #2).
R-04	Fallos en la integración de módulos	Incompatibilidad entre el motor de escaneo, la base de datos y la interfaz de usuario.	Alto	Baja	Implementar pruebas de regresión (Issue #14) después de cada merge en GitHub.

Código	Riesgo	Descripción	Impacto	Probabilidad	Estrategia de Mitigación
R-05	Retrasos en el cronograma	Estimaciones de tiempo inexactas que afecten la entrega del proyecto.	Medio	Media	Seguimiento semanal de avances y uso de metodologías ágiles para la gestión del desarrollo.
R-06	Obsolescencia tecnológica	Cambios en los sistemas operativos objetivo que afecten la compatibilidad.	Medio	Baja	Mantener una arquitectura modular (Issue #5) que permita adaptaciones futuras.

3. Análisis de la Situación actual

3.1. Planteamiento del problema

En la actualidad, los usuarios de sistemas informáticos se enfrentan a un panorama de amenazas digitales cada vez más sofisticadas. Según reportes de seguridad, el número de ataques de malware ha incrementado significativamente en los últimos años, afectando tanto a usuarios domésticos como a organizaciones empresariales.

La problemática principal radica en que muchas soluciones antivirus existentes en el mercado son de pago, lo que limita el acceso de usuarios con recursos limitados a herramientas de protección efectivas. Adicionalmente, las soluciones gratuitas disponibles suelen ofrecer funcionalidades básicas limitadas, sin monitoreo en tiempo real o actualizaciones frecuentes de firmas.

El proyecto SecureGuard Antivirus surge como una respuesta a esta necesidad, ofreciendo una solución de código abierto que permita a los usuarios proteger sus equipos sin incurrir en costos elevados, manteniendo estándares de calidad y funcionalidades competitivas como monitoreo en tiempo real y escaneo basado en firmas.

3.2. Consideraciones de hardware y software

Categoría	Recurso	Especificación	Disponibilidad
Hardware	Estaciones de desarrollo	Procesador Intel Core i5 o superior, 8GB RAM, 256GB SSD	Disponible
	Servidor de pruebas	Máquina virtual con 4GB RAM, 50GB almacenamiento	Disponible (VirtualBox)
	Equipos de prueba	Equipos con Windows 10/11, Linux (Ubuntu)	Disponible
Software	Sistema Operativo	Windows 10/11, Linux (Ubuntu 20.04/22.04)	Disponible
	IDE	Visual Studio Code / PyCharm Community Edition	Disponible (gratuito)
	Lenguaje de programación	Python 3.9+ / C++ para módulos críticos	Disponible (open source)
	Base de Datos	SQLite (embebida) / PostgreSQL	Disponible (open source)
	Control de versiones	Git + GitHub	Disponible (gratuito)
	Gestión de proyectos	GitHub Projects / Trello	Disponible (gratuito)
Infraestructura	Red	Conexión a internet, red LAN	Disponible
	Dominio	No requerido para prototipo	N/A

4. Estudio de Factibilidad

El estudio de factibilidad del proyecto SecureGuard Antivirus se ha realizado siguiendo las directrices establecidas en los issues de la Fase 1, específicamente el Issue #1 (Elaboración de Informe de Factibilidad). Las actividades realizadas para preparar esta evaluación incluyen:

1. Análisis de los requerimientos técnicos y funcionales (Issue #3)
2. Evaluación de recursos disponibles para el desarrollo
3. Identificación de riesgos y estrategias de mitigación
4. Estimación de costos y beneficios del proyecto
5. Análisis del impacto en las diferentes dimensiones (legal, social, ambiental)

El presente informe ha sido aprobado por el equipo de desarrollo y será presentado al docente del curso para su revisión y aprobación.

4.1. Factibilidad Técnica

La viabilidad técnica evalúa si el equipo cuenta con los recursos tecnológicos necesarios para desarrollar e implementar el sistema propuesto.

Evaluación de tecnología actual:

Componente	Tecnología Propuesta	Evaluación
Motor de Escaneo	Python con librerías nativas (<code>os</code> , <code>hashlib</code> , <code>re</code>)	Factible: Python permite manipulación de archivos y cálculos de hash de manera eficiente. Para módulos críticos se puede utilizar C++ mediante extensiones.
Base de Datos de Firmas	SQLite	Factible: Base de datos ligera, embebida, no requiere servidor adicional. Ideal para aplicaciones de escritorio.
Monitoreo en Tiempo Real	<code>watchdog</code> (Python) / <code>inotify</code> (Linux) / <code>ReadDirectoryChangesW</code> (Windows)	Factible: Existen librerías multiplataforma que abstraen las APIs del sistema operativo.
Interfaz de Usuario	Tkinter / PyQt (para versión gráfica) o CLI (para versión consola)	Factible: Ambas opciones son maduras y ampliamente documentadas.
Control de Versiones	Git + GitHub	Factible: Herramientas estandarizadas en la industria.
Entorno de Pruebas	<code>pytest</code> (unitarias), entornos virtuales aislados	Factible: Herramientas gratuitas y robustas.

Hardware requerido:

- Equipos de desarrollo con capacidad para ejecutar entornos virtualizados (pruebas de malware aisladas)
- Almacenamiento suficiente para la base de datos de firmas (aproximadamente 500MB iniciales)

Conclusión Técnica: El proyecto es técnicamente viable. El equipo cuenta con el conocimiento necesario o la capacidad de adquirirlo, y las herramientas tecnológicas seleccionadas son de acceso gratuito y ampliamente soportadas.

4.2. Factibilidad Económica

El propósito del estudio de viabilidad económica es determinar los beneficios económicos del proyecto en contraposición con los costos.

4.2.1. Costos Generales Costos realizados en accesorios y material de oficina necesarios para los procesos.

Concepto	Cantidad	Costo Unitario (S/.)	Costo Total (S/.)
Papel bond A4	1 millar	25.00	25.00
Lapiceros	4	2.50	10.00
Cuaderno de apuntes	2	8.00	16.00
USB 32GB	1	35.00	35.00
Impresiones	50	0.50	25.00
Total			111.00

4.2.2. Costos operativos durante el desarrollo Costos necesarios para la operatividad durante el periodo de desarrollo.

Concepto	Descripción	Costo Total (S/.)
Energía eléctrica	Consumo de equipos durante 16 semanas	120.00
Internet	Conexión para trabajo colaborativo y consultas	200.00
Total		320.00

4.2.3. Costos del ambiente Evaluación de requerimientos técnicos para la implantación.

Concepto	Requerimiento	Costo	Observación
Infraestructura de red	Red LAN existente	S/. 0	Disponible en el entorno universitario/domiciliario
Acceso a Internet	Banda ancha	S/. 0	Ya incluido en costos operativos

Concepto	Requerimiento	Costo	Observación
Máquinas virtuales	VirtualBox (gratis)	S/. 0	Software libre
Total		S/. 0	

4.2.4. Costos de personal Gastos generados por el recurso humano necesario para el desarrollo del sistema.

Rol	Integrante	Horas estimadas	Tarifa por hora (S/.)	Costo Total (S/.)
Jefe de Proyecto / Desarrollador	LLica Mamani, Jimmy Mijair	120	25.00	3,000.00
Desarrollador / QA	Slador Ruiz, Iker Alberto	120	25.00	3,000.00
Total				6,000.00

Nota: Al tratarse de un proyecto académico, este costo representa el valor del trabajo realizado, aunque no implica un desembolso real por parte de la universidad o los estudiantes.

Organización y horario:

Rol	Responsabilidades	Horario semanal
Jefe de Proyecto	Gestión del proyecto, arquitectura, motor de escaneo, integración	8 horas
Desarrollador	Base de datos, interfaz, pruebas unitarias, documentación	8 horas
QA (compartido)	Pruebas funcionales, pruebas de regresión, métricas de calidad	4 horas

4.2.5. Costos totales del desarrollo del sistema

Categoría	Costo (S/.)
Costos Generales	111.00
Costos Operativos	320.00
Costos de Ambiente	0.00
Costos de Personal	6,000.00
Costo Total del Proyecto	6,431.00

Forma de pago: El proyecto se desarrolla en el marco académico, por lo que no se requiere inversión inicial. Los costos generales y operativos son asumidos por los integrantes del equipo.

Conclusión Económica: El proyecto es económicamente viable. Los costos son mínimos (principalmente personal y servicios básicos) y pueden ser asumidos por el equipo de desarrollo.

4.2.6. Análisis de Costos en la Nube (AWS vs Azure) Aunque SecureGuard Antivirus es una aplicación de escritorio que opera localmente, se plantea la posibilidad de una arquitectura híbrida para la distribución de actualizaciones de firmas y telemetría opcional. A continuación se comparan las principales opciones en nube.

Servicios requeridos (escenario de despliegue en nube):

Servicio	Función en el sistema
Almacenamiento de objetos	Alojar base de firmas (<code>malware_hashes.txt</code> , reglas YARA)
CDN	Distribución rápida de actualizaciones de firmas globalmente
Servidor de API REST	Endpoint de actualización y reporte de telemetría (opcional)
Base de datos relacional	Registro de versiones de firmas y estadísticas

Estimación de costos mensuales — Amazon Web Services (AWS):

Servicio AWS	Especificación	Costo Estimado (USD/mes)
Amazon S3	5 GB almacenamiento + 50 GB transferencia	\$1.50
Amazon CloudFront	CDN, 100 GB transferencia	\$9.00

Servicio AWS	Especificación	Costo Estimado (USD/mes)
AWS Lambda	API de actualización (1M invocaciones/mes)	\$0.20
Amazon RDS (db.t3.micro)	PostgreSQL, base de firmas y versiones	\$15.00
Total mensual AWS		\$25.70
Total anual AWS		\$308.40

Estimación de costos mensuales — Microsoft Azure:

Servicio Azure	Especificación	Costo Estimado (USD/mes)
Azure Blob Storage	5 GB + 50 GB transferencia	\$1.20
Azure CDN	100 GB transferencia	\$8.50
Azure Functions	API de actualización (1M ejecuciones/mes)	\$0.20
Azure Database for PostgreSQL (B1ms)	Base de firmas y versiones	\$12.00
Total mensual Azure		\$21.90
Total anual Azure		\$262.80

Comparativa: Despliegue Local vs. Nube:

Criterio	Local (actual)	AWS	Azure
Costo inicial	S/. 0	\$0 (pago por uso)	\$0 (pago por uso)
Costo mensual	S/. 0	~\$25.70	~\$21.90
Escalabilidad	Baja (manual)	Alta (automática)	Alta (automática)
Disponibilidad	Depende del desarrollador	99.99% SLA	99.99% SLA
Tiempo de actualización	Manual (repositorio GitHub)	Automático	Automático
Seguridad de datos	Controlada por el equipo	Gestionada por AWS	Gestionada por Azure

Criterio	Local (actual)	AWS	Azure
Mantenimiento	Alto (gestión propia)	Bajo (managed services)	Bajo (managed services)
Recomendación para v1.0	Adecuado	Para v2.0+	Para v2.0+

Conclusión del análisis de nube: Para la versión académica v1.0, el despliegue local mediante GitHub es suficiente y sin costo. Azure resulta ligeramente más económico para una eventual versión comercial. Se recomienda migrar a nube en la versión v2.0 cuando la base de usuarios justifique la inversión.

4.2.7. Infraestructura como Código (IaC) — Terraform Se presenta un ejemplo de configuración Terraform para el despliegue del servidor de firmas en AWS, listo para escalar SecureGuard Antivirus a producción en la versión v2.0.

```
# =====
# SecureGuard Antivirus - Infraestructura como Código (IaC)
# Proveedor: Amazon Web Services (AWS)
# Versión objetivo: v2.0
# Herramienta: Terraform >= 1.5
# =====

terraform {
  required_version = ">= 1.5"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# --- Configuración del proveedor AWS ---
provider "aws" {
  region = var.aws_region # Región donde se despliegan los recursos (ej. us-east-1)
}

# --- Variables parametrizables ---
variable "aws_region" {
  description = "Región AWS para el despliegue"
  type        = string
}
```

```

    default      = "us-east-1"
}

variable "environment" {
    description = "Entorno de despliegue: dev | staging | prod"
    type        = string
    default      = "prod"
}

variable "db_password" {
    description = "Contraseña de la base de datos de firmas (definir en terraform.tfvars)"
    type        = string
    sensitive    = true # Oculta el valor en los logs de Terraform
}

# --- Bucket S3 para almacenamiento de firmas ---
resource "aws_s3_bucket" "signatures" {
    bucket = "secureguard-signatures-${var.environment}"
    tags = {
        Name           = "SecureGuard Signatures"
        Environment    = var.environment
        Project         = "SecureGuard Antivirus"
    }
}

# Habilitar versionado del bucket (permite rollback de firmas)
resource "aws_s3_bucket_versioning" "signatures_versioning" {
    bucket = aws_s3_bucket.signatures.id
    versioning_configuration {
        status = "Enabled"
    }
}

# Bloquear acceso público al bucket (las firmas se sirven vía CloudFront)
resource "aws_s3_bucket_public_access_block" "signatures_block" {
    bucket                = aws_s3_bucket.signatures.id
    block_public_acls     = true
    block_public_policy   = true
    ignore_public_acls    = true
    restrict_public_buckets = true
}

# --- Distribución CloudFront para CDN de actualizaciones ---
resource "aws_cloudfront_distribution" "signatures_cdn" {
    enabled          = true
    comment          = "CDN para actualizaciones de firmas SecureGuard"

```

```

default_root_object = "malware_hashes.txt"

origin {
    domain_name = aws_s3_bucket.signatures.bucket_regional_domain_name
    origin_id    = "S3-SecureGuard-Signatures"

    s3_origin_config {
        origin_access_identity = aws_cloudfront_origin_access_identity.oai.cloudfront_access_id
    }
}

default_cache_behavior {
    allowed_methods  = ["GET", "HEAD"]          # Solo lectura; las firmas se publican por e
    cached_methods  = ["GET", "HEAD"]
    target_origin_id = "S3-SecureGuard-Signatures"
    viewer_protocol_policy = "redirect-to-https" # Forzar HTTPS para seguridad

    forwarded_values {
        query_string = false
        cookies { forward = "none" }
    }

    min_ttl        = 3600    # Cache mínimo 1 hora
    default_ttl    = 86400   # Cache por defecto 24 horas
    max_ttl        = 604800  # Cache máximo 7 días
}

restrictions {
    geo_restriction { restriction_type = "none" } # Disponible globalmente
}

viewer_certificate {
    cloudfront_default_certificate = true # Usar certificado TLS de CloudFront
}

tags = {
    Environment = var.environment
    Project     = "SecureGuard Antivirus"
}

# Identity para que CloudFront acceda al bucket S3 privado
resource "aws_cloudfront_origin_access_identity" "oai" {
    comment = "OAI para SecureGuard Signatures"
}

```

```

# --- Base de datos RDS PostgreSQL para gestión de versiones de firmas ---
resource "aws_db_instance" "signatures_db" {
  identifier      = "secureguard-signatures-db"
  engine          = "postgres"
  engine_version = "15.4"
  instance_class  = "db.t3.micro" # Instancia de bajo costo para iniciar
  allocated_storage = 20          # 20 GB de almacenamiento inicial

  db_name = "secureguard"
  username = "sgadmin"
  password = var.db_password # Definir en terraform.tfvars (NO commitear)

  skip_final_snapshot = false
  final_snapshot_identifier = "secureguard-final-snapshot"
  deletion_protection = true # Protección contra eliminación accidental

  backup_retention_period = 7 # Respaldo automático por 7 días
  backup_window           = "03:00-04:00" # Ventana de respaldo (UTC)

  tags = {
    Environment = var.environment
    Project      = "SecureGuard Antivirus"
  }
}

# --- Outputs: información relevante tras el despliegue ---
output "cdn_domain" {
  description = "URL del CDN para actualización de firmas (usar en updater.rs)"
  value       = aws_cloudfront_distribution.signatures_cdn.domain_name
}

output "s3_bucket_name" {
  description = "Nombre del bucket S3 para subir nuevas firmas"
  value       = aws_s3_bucket.signatures.id
}

output "db_endpoint" {
  description = "Endpoint de conexión a la base de datos de versiones"
  value       = aws_db_instance.signatures_db.endpoint
  sensitive   = true
}

```

Nota de uso: Para desplegar esta infraestructura ejecutar:

```

terraform init      # Inicializar proveedores
terraform plan      # Revisar cambios antes de aplicar
terraform apply     # Desplegar infraestructura

```

```
terraform destroy          # Eliminar infraestructura (cuando no se necesite)
```

Los valores sensibles (`db_password`) deben definirse en un archivo `terraform.tfvars` que **no debe ser commiteado** al repositorio (agregar al `.gitignore`).

4.3. Factibilidad Operativa

La factibilidad operativa evalúa si los usuarios finales podrán utilizar el sistema y si la organización cuenta con la capacidad para mantenerlo.

Beneficios operativos:

- Protección continua mediante monitoreo en tiempo real sin intervención manual del usuario
- Interfaz intuitiva que permite realizar escaneos programados o manuales con facilidad
- Actualización de la base de datos de firmas para mantener la protección contra nuevas amenazas

Capacidad de mantenimiento:

- El código fuente estará disponible en GitHub, permitiendo su mantenimiento y evolución
- La arquitectura modular (Issue #5) facilita la actualización de componentes individuales
- La documentación generada en las fases del proyecto permitirá comprender el funcionamiento interno

Interesados identificados:

Interesado	Rol	Expectativas
Usuarios finales	Utilizadores del antivirus	Protección efectiva, bajo consumo de recursos, fácil uso
Docente del curso	Evaluador	Cumplimiento de requisitos, calidad del producto
Equipo de desarrollo	Creadores	Aprendizaje, entrega exitosa del proyecto
Futuros contribuyentes	Desarrolladores open source	Código mantenible, documentación clara

Conclusión Operativa: El proyecto es operativamente viable. Los usuarios finales podrán utilizar el sistema sin necesidad de conocimientos técnicos avanzados, y el equipo de desarrollo cuenta con la capacidad para mantenerlo.

4.4. Factibilidad Legal

Determina si existe conflicto del proyecto con restricciones legales.

Aspecto Legal	Evaluación
Propiedad intelectual	El software se desarrollará como código abierto. Se utilizarán licencias compatibles (MIT, GPL) para los componentes de terceros. No se infringirán derechos de autor.
Protección de datos	El antivirus procesa archivos locales del usuario. No se recopilan ni almacenan datos personales sin consentimiento. Cumple con la Ley de Protección de Datos Personales (Ley N° 29733).
Regulaciones de software de seguridad	El software no utiliza técnicas de ingeniería inversa prohibidas. Su funcionamiento se limita al escaneo y eliminación de malware en el equipo del usuario.
Criptografía	Si se implementan funciones hash (MD5, SHA) para identificación de firmas, estas son legales y de uso permitido en el Perú.

Conclusión Legal: El proyecto es legalmente viable. No se identifican conflictos con las leyes y regulaciones peruanas.

4.5. Factibilidad Social

Evaluación de influencias y asuntos de índole social y cultural.

Aspecto Social	Evaluación
Impacto social	El proyecto proporciona una herramienta de seguridad gratuita, beneficiando a usuarios con recursos limitados que no pueden acceder a soluciones comerciales.

Aspecto Social	Evaluación
Código de conducta	El equipo de desarrollo se regirá por un código de conducta que promueve la colaboración respetuosa y la inclusión.
Accesibilidad	La interfaz se diseñará considerando principios básicos de accesibilidad para usuarios con discapacidades visuales o motrices.
Educación digital	El proyecto contribuye a la formación de los estudiantes en temas de seguridad informática y calidad de software.

Conclusión Social: El proyecto es socialmente viable, con un impacto positivo al ofrecer una herramienta de seguridad gratuita y contribuir a la formación profesional.

4.6. Factibilidad Ambiental

Evaluación del impacto y repercusión en el medio ambiente.

Aspecto Ambiental	Evaluación
Consumo energético	El software optimizado minimizará el consumo de recursos del equipo, reduciendo indirectamente el consumo energético.
Residuos electrónicos	Al ser software, no genera residuos físicos. Su distribución digital evita el uso de materiales físicos.
Ciclo de vida	El código abierto permite extender la vida útil del software, evitando la obsolescencia programada.
Infraestructura	Se utiliza infraestructura existente, sin necesidad de nuevo hardware.

Conclusión Ambiental: El proyecto es ambientalmente viable. Su naturaleza digital y su enfoque en la eficiencia contribuyen a un impacto ambiental reducido.

5. Análisis Financiero

5.1. Justificación de la Inversión

5.1.1. Beneficios del Proyecto Beneficios Tangibles:

Beneficio	Descripción	Estimación Anual (S/.)
Ahorro en licencias	Los usuarios no necesitan adquirir antivirus comerciales	200 por usuario
Reducción de incidentes de seguridad	Menor probabilidad de infecciones que requieran soporte técnico	500 por incidente evitado
Eficiencia operativa	Automatización del escaneo y monitoreo, reduciendo intervención manual	100 por usuario

Beneficios Intangibles:

- Mejora en la seguridad de la información de los usuarios
- Contribución a la comunidad de código abierto
- Desarrollo de competencias profesionales en el equipo
- Disponibilidad de una herramienta de seguridad gratuita y confiable
- Toma acertada de decisiones mediante reportes de escaneo
- Aumento en la confiabilidad de la información procesada
- Logro de ventajas competitivas en el ámbito académico

5.1.2. Criterios de Inversión Para la evaluación financiera se considera un horizonte de 3 años, con un costo de oportunidad de capital (COK) del 12% anual.

Proyección de beneficios anuales (para 100 usuarios):

Año	Beneficios Estimados (S/.)	Costos de Mantenimiento (S/.)	Beneficio Neto (S/.)
0	0	6,431	-6,431
1	20,000	500	19,500
2	25,000	600	24,400
3	30,000	700	29,300

Nota: Los beneficios estimados consideran ahorro en licencias y reducción de incidentes para una base de 100 usuarios.

5.1.2.1. Relación Beneficio/Costo (B/C) Valor Presente de los Beneficios (VPB): - Año 1: $19,500 / (1.12)^1 = 17,410.71$ - Año 2: $24,400 / (1.12)^2 = 19,450.76$ - Año 3: $29,300 / (1.12)^3 = 20,851.23$ - **Total VPB = 57,712.70**

Valor Presente de los Costos (VPC): - Año 0: $6,431 / (1.12) = 6,431.00$ - Año 1: $500 / (1.12)^1 = 446.43$ - Año 2: $600 / (1.12)^2 = 478.30$ - Año 3: $700 / (1.12)^3 = 498.25$ - **Total VPC = 7,853.98**

Relación B/C = VPB / VPC = 57,712.70 / 7,853.98 = 7.35

Interpretación: $B/C > 1$, por lo tanto, el proyecto se acepta.

5.1.2.2. Valor Actual Neto (VAN) $VAN = -6,431 + 19,500/(1.12)^1 + 24,400/(1.12)^2 + 29,300/(1.12)^3$

$VAN = -6,431 + 17,410.71 + 19,450.76 + 20,851.23$

VAN = S/. 51,281.70

Interpretación: $VAN > 0$, por lo tanto, el proyecto se acepta.

5.1.2.3. Tasa Interna de Retorno (TIR) La TIR se calcula como la tasa que hace el VAN igual a cero:

$VAN = 0 = -6,431 + 19,500/(1+TIR)^1 + 24,400/(1+TIR)^2 + 29,300/(1+TIR)^3$

Resolviendo la ecuación:

TIR 185%

Interpretación: $TIR (185\%) > COK (12\%)$, por lo tanto, el proyecto se acepta.

Resumen de Criterios de Inversión:

Indicador	Valor	Criterio	Decisión
Relación B/C	7.35	> 1	Aceptar
VAN	S/. 51,281.70	> 0	Aceptar
TIR	185%	$> COK (12\%)$	Aceptar

6. Conclusiones

Luego de realizar el análisis exhaustivo de factibilidad en sus diferentes dimensiones, se presentan las siguientes conclusiones:

1. **Factibilidad Técnica:** El proyecto es técnicamente viable. Se cuenta con las herramientas y conocimientos necesarios para desarrollar el motor de escaneo, la base de datos de firmas, el monitoreo en tiempo real y la

interfaz de usuario. Las tecnologías seleccionadas (Python, SQLite, Git) son de acceso gratuito y ampliamente soportadas.

2. **Factibilidad Económica:** El proyecto es económicamente viable. Los costos totales estimados ascienden a S/. 6,431.00, siendo mayormente costos de personal (trabajo académico). Los costos generales y operativos son mínimos y pueden ser asumidos por el equipo de desarrollo.
3. **Factibilidad Operativa:** El proyecto es operativamente viable. Los usuarios finales podrán utilizar el sistema sin dificultad gracias a una interfaz intuitiva. El equipo de desarrollo cuenta con la capacidad para mantener el sistema y la arquitectura modular facilitará futuras actualizaciones.
4. **Factibilidad Legal:** El proyecto cumple con las disposiciones legales aplicables, incluyendo la Ley de Protección de Datos Personales y las normativas de propiedad intelectual. Se utilizarán licencias de código abierto compatibles.
5. **Factibilidad Social:** El proyecto tiene un impacto social positivo al ofrecer una herramienta de seguridad gratuita, beneficiando a usuarios con recursos limitados y contribuyendo a la formación profesional de los estudiantes.
6. **Factibilidad Ambiental:** El proyecto es ambientalmente viable. Su naturaleza digital evita la generación de residuos físicos, y su enfoque en la eficiencia contribuye a un menor consumo energético.
7. **Análisis Financiero:** Los indicadores financieros ($B/C = 7.35$, $VAN = S/. 51,281.70$, $TIR = 185\%$) demuestran que el proyecto es rentable y genera valor, incluso considerando un escenario conservador de beneficios.

Conclusión Final:

El proyecto **SecureGuard Antivirus** es **VIABLE y FACTIBLE** en todas sus dimensiones. Se recomienda proceder con el desarrollo siguiendo el cronograma establecido en los issues del proyecto, iniciando con la definición de la visión y alcance (Issue #2) y la especificación detallada de requerimientos (Issue #3).